Sensors and Actuators (Sensorik und Aktorik) — 10-Day Edition

Day 1 · Introduction, Pre-Study Rehash & Basic Electronics

Make the physical world legible — turn a real quantity into a trustworthy number, and that number into light, motion, or heat.

whoami

ULrich NOrbisrath (ulno) — ulno.net

- Educator · Consultant · Mentor
- Researcher · Inventor · Maker · Artist
- YouTuber (youtube.ulno.net) → Geek
- **Globalist** — lived, taught, and researched in Estonia · USA · Germany · Austria · Kazakhstan · Singapore · Indonesia · Brazil
- **Research:** VR/AR · Internet of Things · Digital Twin · Software Craftsmanship · Education & Creativity
- **PhD: Home Automation** (RWTH Aachen)
- Around here: IoTempire and IoTempower

Who are U?

*(Write this down — it is the **start of your in-class portfolio notes**, in addition to Module 0. Note the sub-questions as we interact.)*

- Who has **programming** experience? In which languages?
- Who has used a **microcontroller** — Arduino, ESP32/ESP8266, Raspberry Pi, …?
- Who has wired a **breadboard**, a resistor, an LED, or a button? Who has **soldered**?
- Who has used a **sensor** or an **actuator** before? Which ones?
- Who has used a **multimeter**? An **oscilloscope** or **logic analyzer**?
- Who has used **Git / GitHub**? The **command line**? **Python**?
- Who has **3D-printed**, laser-cut, or built a physical prototype?
- **What would you like to measure or move** in this class?
- **What are your expectations?** What would you like to build?

This is an experiment!

- Teaching with **modern education principles** — challenge- and project-based
- You are supposed to **learn and explore**
- Put **you** front and center; give the content **meaning**
- **You** bring a meaningful challenge

Class logistics & assessment (short version)

- **No separate exercise sheets — but expect at least ~6 hours/week of self-study** to finish labs, portfolio, reflections, and project work
- **10 working days / 12 four-hour sessions (48 contact hours)**, plus optional tutorial support announced via the LMS
- **You work in small Task Pods — pairs by default** (a third podmate only by arrangement). For the **final project, two pods merge into a Project Team (4–6)**.
- In-class exercises are initially **individual**
- **Ongoing feedback** in the lab — talk to us!
- Final assessment: **module points + final project + reflections** (+ bonuses), with your **portfolio** as the record
- **~10 minutes of reflection after each meeting — evening or next morning, not during the 4-hour studio**
- **Checkpoints** are short portfolio conversations (10–12 minutes), not task-completion audits
- The details live in the **syllabus** — next slide

Everything lives in the online workbook

Central entry point: **the course repository**.

- **Module index & roadmap** — every module and the day-by-day plan
- **Syllabus** — schedule, learning objectives, assessment, tools, policies
- **Pre-study guide** — Module 0
- **Module 1** — today's studio
- **Resource bank** — datasheets, cheat sheets, troubleshooting, instructor hints

(Let's open the workbook on screen now — this is the material we will actually use.)

How the class works — the deal we make

- Every module **begins with its learning goals** — that is the *contract*: demonstrate the goals and you have met the module
- The **tasks are a draft** — you may modify, replace, or extend them, as long as you reach the goals
- Tasks are marked ★ **Core** or ◇ **Stretcher** — the stretcher is the natural trim point when time is short
- A moving ***DRAFT BOUNDARY*** marks what we are still designing together — your input shapes it
- **AI is a tool you verify**, not an oracle: draft with it, then check the datasheet, the multimeter, and your measurement
- **Already know basic electronics?** You may skip that task — document why in the portfolio and show the key skills

(Let's open the syllabus and Module 1 together now.)

Today's path

1. **Who we are** — and how this class works
2. **Rehash the pre-study** — chain, ADC, accuracy, datasheets, actuators, project ideas
3. **Form Task Pods**
4. **Basic electronics** — LED/relay as actuator, button with pull-up/pull-down, contact bounce
5. **Arduino IDE + serial console** — inspect and plot data
6. **ADC → LDR → first filtering** — the measurement chain in practice

(No networking today — that comes later.)

Form your Task Pod — who do you build with?

1. On a note, write **your skills** (languages, tools, hardware, design, …) and **your expectations / what you want to build**.
2. No pod yet? Find **one other unpaired person**; a third podmate is only by arrangement. Compare notes and check whether you **complement** each other — not just “same as me”.
3. **You will work in a Task Pod — pair by default** (a third podmate only by arrangement). Exchange contacts and a first idea today.
4. Very optional (more chances later — even to change your initial pod): pair your pod with a **neighbouring pod** for a **2-pod practice round** — present your learning-goal proof and give each other free-style feedback.
5. For the **final project, two pods merge into a Project Team of 4–6** — your neighbouring pod is the natural partner.

Portfolio check — before we touch hardware

1. Open your portfolio repository (created from the iot-portfolio-template; instructor added as collaborator).
2. Add a note for today: `Module01/who-am-i.md` (or a section in `Module01/README.md`) with your **Who are U?** answers and expectations.
3. Add your **pod** names, skills, and first idea.
4. Commit and push — a small commit with a clear message.
5. Keep the pre-study where it is: this is an **additional** in-class entry, not a replacement.

Pre-study check-in

(In your pod. Two minutes per person.)

1. Show **one artefact** from Module 0: your chain diagram, Wokwi screenshot, datasheet note, or project abstract.
2. Explain **one thing you understood only after doing it**.
3. Name **one question you still have** — park it on the board.

Then we rehash the big ideas together.

Rehash: draw the chain

(In your pod — two minutes.)

Pick **one device you actually own** and draw:

physical quantity → transducer → conditioning → ADC + MCU → system → actuator

- Name **each stage**.
- Mark **one place where error or noise enters**.
- Where is the number converted? Where is it filtered? Where does an actuator get commanded?

Rehash: the chain you should be able to draw

Physical quantity → transducer → conditioning → ADC + MCU → system → actuator

- **Transducer:** converts the physical quantity into voltage, resistance, charge, current, or frequency.
- **Conditioning:** divider, amplifier, filter, reference — make the signal match the ADC.
- **ADC + MCU:** sample, quantize, code, filter, calibrate.
- **System:** logging, visualization, control logic.
- **Actuator:** commanded via PWM/driver; it acts on the world.

One-line takeaway: a sensor is not a number; it is a physical effect, conditioned, converted, coded, and interpreted.

Rehash: ADC and the Wokwi observation

- An **n-bit ADC** gives **`2^n` codes** over the reference range:
  - 8 bit → 256 · 10 bit → 1,024 · 12 bit → 4,096
- With `Vref = 3.3 V`, a 12-bit step is `3.3 V / 4096 ≈ 0.8 mV`.
- Changing the resolution changes the **step size and the number of codes**.
- It does **not** automatically change **accuracy** — offset, gain, noise, and nonlinearity still decide how close you are to the true value.

The ESP32 trap: its 12-bit ADC is much more precise in *resolution* than it is accurate near the rails. You will calibrate around this in Module 3.

Rehash: accuracy vs. precision

- **Accuracy** — closeness to the **true value**.
- **Precision** — **repeatability** of repeated measurements.
- **Resolution** — the smallest **reported change**; not the same as accuracy.

Examples:

- **Precise but inaccurate:** a scale that always reads 2 kg too high.
- **Accurate but imprecise:** a cheap thermometer that scatters around the true value but averages correctly.

A number without units, conditions, and uncertainty is not a measurement.

Rehash: datasheets and I²C

For each sensor datasheet, find the **conditions** behind every performance number:

- Which **physical quantity**, by which **principle**?
- Which **interface** — analog, I²C, SPI, PWM, OneWire?
- Which **address** (if I²C)?
- Range, resolution, accuracy, response time, temperature dependence.

I²C preview: many devices share the same two wires — **SDA** and **SCL**. The **7-bit address** is how a device knows a message is for it.

- Two devices with the **same address** conflict.
- Solutions: change an address pin/jumper, or use an **I²C multiplexer** (Module 2).

Rehash: choosing an actuator

The problem: move a small door latch **once every few minutes**.

- **Good fit:** a **servo** or a **solenoid/linear actuator** — simple positioning or a short push/pull, easy to command, low duty cycle.
- **Poor fit:** a **bare DC motor** — it spins, but without feedback or a mechanism it does not position the latch; it also needs a driver.
- **Overkill:** a stepper, unless you need precise incremental motion.

Justify by torque, control simplicity, and power. And remember: **logic pins command; drivers deliver current.**

Rehash: LED power math

A 30-LED WS2812 strip at full white, ~60 mA per LED:

```
30 × 60 mA = 1,800 mA = 1.8 A
```

Add margin → plan for a **≥ 2 A 5 V supply**.

- The microcontroller's **5 V pin/regulator cannot supply this**.
- Use a **separate 5 V supply**, a **common ground**, and **power injection** for long strips.
- Data still comes from the MCU — but the LEDs get their current from the supply.

Logic pins command. They do not power motors, strips, relays, or locks.

Rehash: your project abstract

(One sentence each, in your pod.)

Who is the stakeholder — a person with a name and a problem — and **what do they need**?

Then add:

- **two sensors** you would start with, and
- **one actuator** that responds.

Examples from the pre-study: liquid measurement in a forest · people counter · flow sensor with light barriers · Hall-effect power monitor · interactive LED installation · music-synchronized fountain.

This is a direction, not a contract. It may change.

From pre-study to practice: safety and kit

- All boards here are **3.3 V logic** — never put **5 V** on a 3.3 V input.
- **Never connect 3.3 V directly to GND.**
- Every circuit needs a **common ground**.
- **Disconnect power before rewiring.**
- Every LED needs a **current-limiting resistor**.
- Relays and motors need a **driver/module** and often their **own supply**.
- In this lab: **low-voltage loads only — never switch mains.**

Kit for today: board, USB cable, breadboard, jumper wires, LED + 220–330 Ω resistor, button, 10 kΩ resistor (or internal pull-up), 100 nF capacitor (optional), potentiometer, LDR.

Workbook tasks: where to start

Open **Module 1** in the workbook and work through it in order:

1. **Task 0 — Form your Task Pod** (done on the previous slide)
2. **Task 1 — Basic Electronics** — LED/relay, button pull-up/pull-down, bounce
3. **Task 2 — Arduino IDE, Serial Console, and Visualization**
4. **Task 3 — The ADC Precision Dance** (potentiometer)
5. **Task 4 — LDR** (analog light sensor)
6. **Task 5 — First Filtering** (average, median, cutoff)

Skip Task 1 if you already know it — add a portfolio note explaining where you learned it, and show the key skills in a quick instructor check.

Task 1 — what “done” looks like

- **Actor:** LED lights through a calculated resistor; or a relay module switches a low-voltage load safely.
- **Digital sensor:** button reads reliably with a **defined pull-up or pull-down** — no floating input.
- **Bounce:** you can show **one press producing multiple edges**.
- **Mitigation:** software debounce works; optionally compare with an **RC** (`100 nF` + `10 kΩ`) or a **Schmitt trigger**.
- **Evidence:** photo/schematic, resistor calculation, pull-up/pull-down explanation, before/after counts.

(If you skip: prior experience + quick check, documented.)

Task 2 — what “done” looks like

- Arduino IDE 2 (or PlatformIO) uploads a sketch to **your** board.
- **Serial Monitor** shows structured, readable output.
- **Serial Plotter** shows at least one signal — or your preferred local plotting tool, documented.
- You can explain what `delay()` does to your sampling loop and why `millis()` is often better.
- **Evidence:** serial log/screenshot, plot, tool choice, loop-timing note.

The baseline is the Arduino IDE. If you already use Teleplot or a Python plotter, keep it — just document it.

Today's exit ticket

Before you leave, commit to your portfolio:

- ☐ Your **“Who are U?”** notes from today
- ☐ Your **pod** (names, skills, first idea)
- ☐ One **photo or schematic** and your **resistor calculation**
- ☐ Your **pull-up/pull-down** explanation
- ☐ Your **bounce counts** before/after debouncing
- ☐ At least one **serial log or plot**

(Small commits, clear messages — the portfolio is the evidence.)

Where we go next

- **Module 2:** build/choose our own ESP32 scope/logic analyzer, then I²C and sensor addressing — scan the bus, read registers, resolve address conflicts.
- **Before next session:** finish the basic-electronics and serial tasks, commit your evidence, and bring your open questions.
- **Checkpoint 1:** start of Day 4, covering Modules 1–3.

Questions?